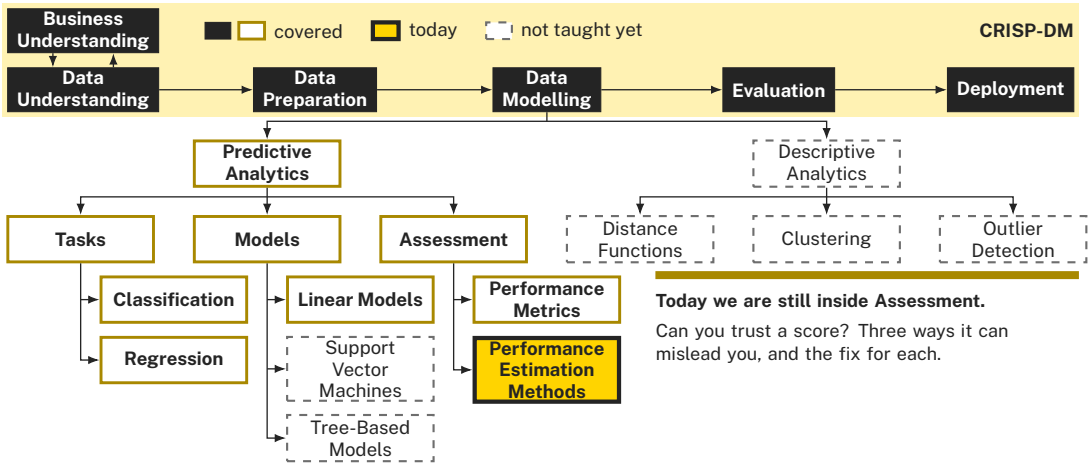
DALHOUSIE UNIVERSITY

Performance Estimation (cont.)

Time-ordered data, nesting, and comparing models

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where Performance Estimation Sits



Today we are still inside Assessment.

Can you trust a score? Three ways it can mislead you, and the fix for each.

Can You Trust the Score?

Holdout, random subsampling, cross-validation and the bootstrap turn a model into a score. Here are three ways that score can fool you.

1

The split saw the future

Shuffled folds let a time series train on days that come after its test days.

2

The search got lucky

Try 16 values of k and keep the best score, and part of that score is luck.

3

The winner won by luck

On handwritten digits, 5-NN beats LDA by 2.4 points. Is that real, or luck?

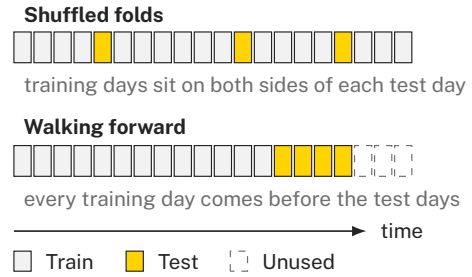
The Split Saw the Future

Shuffled folds let a model study tomorrow before it is tested on today.

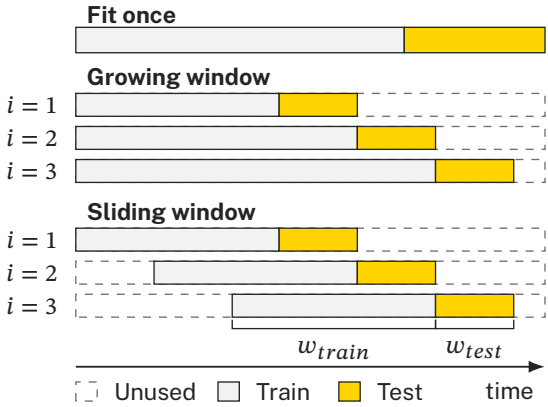
Shuffling Leaks the Future

- Cross-validation and the bootstrap draw rows at random, which assumes the rows have no order.
- In a time series each value depends on the ones before it. Shuffled, a test day such as Thursday can sit between training days Wednesday *and* Friday.
- The model is graded on a question whose answer it has partly seen, so the score looks better than any real forecast can be.

Train only on the past. Test only on the future.



Three Ways to Walk Forward



Fit once

Fit one model on the past and use it for the whole test period. It never learns from newer data.

Growing window

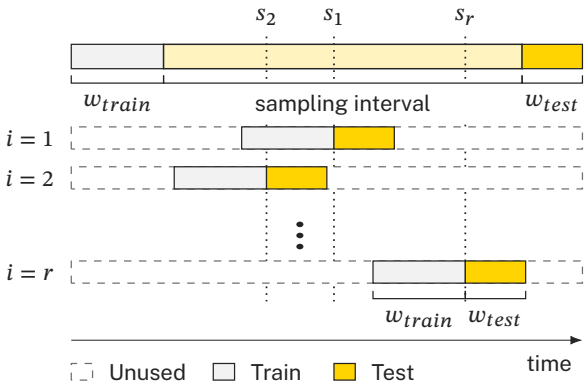
After every test block, refit on *all* the data so far. It assumes the pattern is stable.

Sliding window

After every test block, refit on the most recent rows only. It assumes the pattern drifts.

You choose w_{train} and w_{test}. The test-block scores are averaged.

Random Subsampling Along Time



Random subsampling, as previously seen, but every split trains before it tests.

1. Fix a training length w_{train} and a test length w_{test}.
2. Pick r random cut points, each with room for both windows.
3. At each cut, train on the w_{train} rows just before it and test on the next w_{test} rows.
4. Average the r test scores.

How Much Does Shuffling Flatter?

DALHOUSIE UNIVERSITY

- 200 simulated days. Each day's value is the previous one plus a random step, a **random walk**.
- The model is 5-NN regression with the day number as its only predictor, so it can only copy the values of nearby days. It is extreme on purpose, to make the leak visible.
- Two estimates of its error. Shuffled 10-fold cross-validation, and a sliding window that trains on 100 days and tests the next 10, in 10 test blocks.
- The score is the mean squared error, lower is better.

Mean squared error	R	Python
Shuffled 10-fold	0.78	0.75
Sliding window	6.37	8.46

Shuffled, every test day has training days on both sides, and copying them is nearly right. Walking forward, the nearest training days are all in the past, and the walk drifts away from them.

R and Python simulate different walks, so compare the pattern.

Shuffled Folds Against Walking Forward

R

```r
set.seed(3141)                  # a random walk
n <- 200
d <- data.frame(t = 1:n, y = cumsum(rnorm(n)))
mse <- function(tr, te) {       # kNN on the date
  f <- knnreg(y ~ t, d[tr, ], k = 5)
  mean((predict(f, d[te, ]) - d$y[te])^2)
}
shuffled <- createFolds(d$y, k = 10)
forward <- createTimeSlices(1:n,
  initialWindow = 100, horizon = 10,
  fixedWindow = TRUE, skip = 9)   # sliding
mean(sapply(shuffled,
  function(te) mse(-te, te)))     # 0.78
mean(mapply(mse, forward$train,
  forward$test))                  # 6.37
```

Python

```python
rng = np.random.default_rng(3141)  # a walk
t = np.arange(1, 201).reshape(-1, 1)
y_t = np.cumsum(rng.normal(size=200))
knn_t = KNeighborsRegressor(5)      # on the date
def mse(cv):
    return -cross_val_score(knn_t, t, y_t, cv=cv,
        scoring="neg_mean_squared_error").mean()
print(mse(KFold(10, shuffle=True,
                random_state=3141)))   # 0.75
fwd = TimeSeriesSplit(10, test_size=10,
                      max_train_size=100)
print(mse(fwd))                        # 8.46
```

In R, `initialWindow` is w_{train}, `horizon` is w_{test}, and `skip = 9` moves 10 days between blocks. In Python, `test_size` is w_{test} and `max_train_size` makes the window slide.

The Search Got Lucky

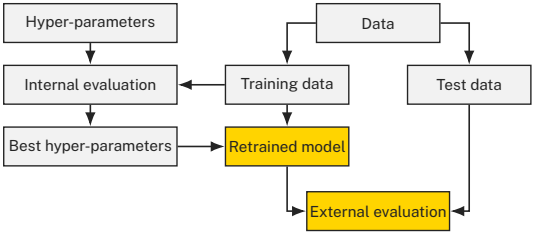
The best of sixteen noisy scores is partly the luckiest of sixteen.

The Luckiest of Sixteen

Sixteen students guess every answer on a quiz. The top score is well above the average, yet nobody knows more than anyone else.

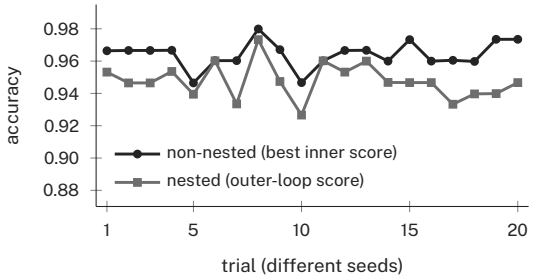
A grid search is the same. Cross-validation on the training rows gives each of 16 values of k an **inner** score, and the search keeps the best. That best score is partly the best setting and partly the luckiest folds.

Split the rows again, **outside** the search. Run the whole search on each outer training part, score its choice on rows it never saw, and report that outer score.



Nesting, as previously seen. Choose inside, score outside.

How Much Luck Was in the Best Score?



Standardized kNN on `iris`, 16 values of k, 4 inner folds to choose, 4 outer folds to score, repeated over 20 seeds.

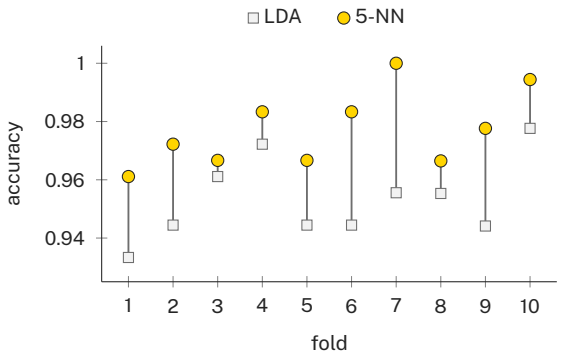
- Best inner score, mean 0.964.
- Nested outer score, mean 0.948.
- The inner score is higher on 19 of 20 seeds, by 1.64 accuracy points on average and up to 3.36.

A score can also look better by chance alone, even with no search at all. That is trap 3.

The Winner Won by Luck

A higher score is not yet a better model. First ask whether luck explains the gap.

5-NN Beat LDA on All Ten Folds



`digits`, 1797 handwritten digits stored as 8×8 pixel images, so 64 predictors and 10 classes. The ten folds are made once, with a fixed seed, and given to both models, so each fold is a head-to-head on the same test rows.

Mean accuracy 0.977 for 5-NN and 0.953 for LDA, a gap of 2.4 points.

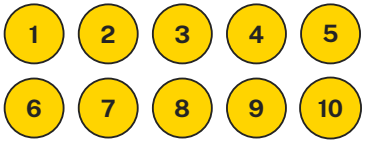
5-NN won every fold. Is that proof, or could it be luck?

If the Two Were Equally Good

DALHOUSIE UNIVERSITY

1. Suppose the two models are equally good. This is the **null hypothesis**.
2. Then each fold is a coin flip, and either model wins half the time.
3. All ten folds going to 5-NN has probability $(1/2)^{10} = 1/1024$. All ten going to LDA would surprise us as much, so $p = 2/1024 = 0.002$.
4. That probability is the **p-value**, how surprising the result would be if there were no real difference.

Below a threshold chosen in advance, usually 0.05, luck is a poor explanation and the gap is called **significant**.



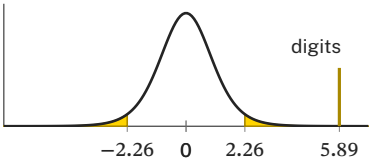
ten folds, ten 5-NN wins
$p = 0.002$ if each fold were a fair coin

Weighing the Size of the Gaps

Coin flips count wins and ignore their size. The **paired t-test** uses the gaps d_i, 5-NN's accuracy minus LDA's on fold i.

$$t = \frac{\text{mean of the gaps}}{\text{SD of the gaps}/\sqrt{n}}$$

The bottom is the standard error of the mean gap, how far luck alone would move it. On `digits`, $t = 0.02392/(0.01285/\sqrt{10}) = 5.89$.



t if the models were equally good

If the models were equally good and the gaps independent, t would follow this curve, the t distribution with $n - 1 = 9$ **degrees of freedom**. The shaded tails beyond ± 2.26 hold 5%. 5.89 lies far out, so $p = 0.0002$, the area beyond it, which `t.test` and `ttest_rel` compute.

What Significance Does Not Mean

DALHOUSIE UNIVERSITY

A small p-value says

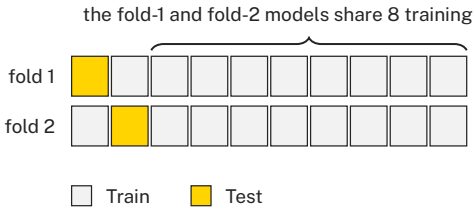
- If the models were equally good, a gap this large would be rare.
- So luck is a poor explanation, *provided the test's assumptions hold*.

A small p-value does not say

- That the gap is large. A tiny gap becomes significant with enough rows.
- That the null hypothesis has probability p of being true.
- That the winner wins elsewhere. One data set tests that data set.

Report the size of the gap beside the p-value.

Ten Folds Are Not Ten Coins

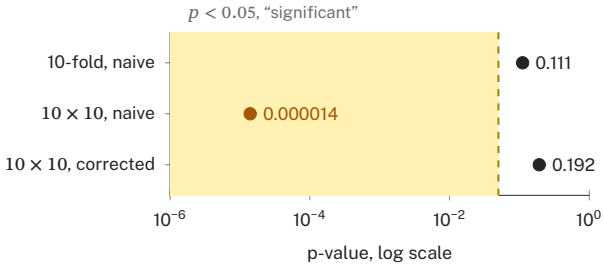


- Coin flips are independent. One flip says nothing about the next.
- Folds are not. Any two training sets in 10-fold CV share 8 of their 9 blocks, so the ten fold models are near copies.
- When one fold favours 5-NN, the others tend to as well, so ten folds carry less evidence than ten coins.

A test that treats folds as independent is overconfident.

Repeating the CV Fakes Significance

A closer race. On `breast_cancer`, 569 biopsies with 30 measurements each, 5-NN beats LDA by 1.2 accuracy points.



- One 10-fold run, paired t-test, $p = 0.111$. Not significant.
- Repeat it ten times and test the 100 gaps as if independent, $p = 0.000014$.
- Why. The standard error divides by $\sqrt{n}$, which grows from 3.2 to 10, on the same 569 rows.

Repeating cross-validation makes the estimate steadier. It does not add evidence.

Pay for the Overlap

Nadeau and Bengio's **corrected resampled t-test** keeps the paired t-test and widens its standard error to pay for the overlap.

$$s\sqrt{\frac{1}{n}} \quad \text{becomes} \quad s\sqrt{\frac{1}{n} + \frac{n_{test}}{n_{train}}}$$

s is the SD of the gaps and n their number. n_{test}/n_{train} is the number of test rows per training row in one split.

- In K-fold CV each fold tests 1 row for every $K-1$ it trains on, so the term is $1/9$ for 10 folds and $1/4$ for 5.
- Repeating the CV grows n and shrinks $1/n$. The overlap term never shrinks, so repetition no longer buys significance.
- The new t is read against the same t distribution, with $n-1$ degrees of freedom.

The Corrected Test on Both Data Sets

Only the standard error changes, so the corrected t is the naive t times a fixed factor.

$$t_{corrected} = t_{naive} \times \sqrt{\frac{1/n}{1/n + n_{test}/n_{train}}}$$

5-NN against LDA	naive t	factor	corrected t	p
`digits`, 10-fold	5.89	0.69	4.05	**0.003**
`breast_cancer`, 10×10	4.58	0.29	1.32	**0.192**

On `digits` the gap survives, so luck is an unlikely explanation for 5-NN's lead. On `breast_cancer`, 100 repeated folds bought nothing, and the 1.1-point gap could be luck.

One Set of Folds for Every Model

R

```r
set.seed(3141)
folds <- createFolds(iris$Species, k = 10,
                     returnTrain = TRUE)
ctrl  <- trainControl("cv", index = folds)
m_knn <- train(Species ~ ., iris, "knn",
               preProcess = c("center", "scale"),
               tuneGrid = data.frame(k = 5),
               trControl = ctrl)
m_lda <- train(Species ~ ., iris, "lda",
               trControl = ctrl)
identical(m_knn$control$index,
          m_lda$control$index)           # TRUE
rs <- resamples(list(kNN = m_knn,
                     LDA = m_lda))
# resamples() matches fold labels only
v <- rs$values
mean(v[["kNN~Accuracy"]] - v[["LDA~Accuracy"]])
# -0.027, LDA ahead on these folds
```

Python

```python
X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(10, shuffle=True,
                     random_state=3141)
# ONE splitter object, reused for every workflow
a = cross_val_score(knn5, X, y, cv=cv)
b = cross_val_score(lda,  X, y, cv=cv)
# WRONG: a fresh partition per call, no warning
# cross_val_score(knn5, X, y,
#     cv=StratifiedKFold(10, shuffle=True))
splits = list(cv.split(X, y))  # safest: freeze
a = cross_val_score(knn5, X, y, cv=splits)
print(a[:3].round(4))   # 0.9825 0.9474 0.9474
print((a - b).mean().round(4))   # 0.0123
```

`knn5` is a standardized 5-NN pipeline and `lda` an LDA model. Folds drawn separately pair nothing. R uses `iris` for the same recipe.

Testing Two Models, the Naive Tests

R

```r
n <- c(rep(180, 7), rep(179, 3))  # test images
knn <- c(173, 175, 174, 177, 174,
         177, 180, 173, 175, 178) / n
lda <- c(168, 170, 173, 175, 170,
         170, 172, 171, 169, 175) / n
d <- knn - lda
binom.test(sum(d > 0), length(d))
#   p-value = 0.001953, coin flips (naive)
t.test(knn, lda, paired = TRUE)
#   t = 5.8891, p-value = 0.0002322 (naive)
```

Python

```python
n = np.array([180] * 7 + [179] * 3)  # test images
knn = pd.Series([173, 175, 174, 177, 174,
                 177, 180, 173, 175, 178]) / n
lda = pd.Series([168, 170, 173, 175, 170,
                 170, 172, 171, 169, 175]) / n
d = knn - lda
print(stats.binomtest(int((d > 0).sum()),
                      len(d)).pvalue)
# 0.001953, ten coin flips (naive)
print(stats.ttest_rel(knn, lda))
# statistic 5.889, pvalue 0.000232 (naive)
```

Both languages type in the same ten `digits` folds, the correct predictions of 5-NN and LDA out of 179 or 180 test images, so they test identical numbers. Both tests here ignore the overlap between training sets.

Testing Two Models, the Corrected Test

R

```r
corrected_t <- function(d, ratio) { # test/train
  v <- (1 / length(d) + ratio) * var(d)
  t <- mean(d) / sqrt(v)
  c(t = t, p = 2 * pt(-abs(t), length(d) - 1))
}
corrected_t(d, 1 / 9)     # t 4.053, p 0.00287
```

Python

```python
def corrected_t(d, ratio):  # n_test / n_train
    v = (1 / len(d) + ratio) * d.var(ddof=1)
    t = d.mean() / np.sqrt(v)
    return t, 2 * stats.t.sf(abs(t), len(d) - 1)
print(corrected_t(d, 1 / 9))     # 4.053 0.00287
```

The same differences `d`. Only `corrected_t` pays for the overlap, through the test-to-train ratio $1/9$ of 10-fold cross-validation, and gives $p = 0.003$.

Three Traps, Three Fixes

DALHOUSIE UNIVERSITY

1

The split saw the future

Shuffled folds train a time series on its own future.

Fix. Fit once, growing, sliding or random windows, always training on the past.

2

The search got lucky

The best inner score of a search is biased upward.

Fix. Report the outer score of a nested search.

3

The winner won by luck

Overlapping folds make naive tests overconfident.

Fix. Same folds for every model, then the corrected resampled t-test.

A p-value measures surprise under an assumption. Report the size of the gap beside it. Next, a support vector machine is tuned inside cross-validation and compared with kNN on the same folds.